

Name: Aarhan

Reg: 192525301

Course: compiler design
CSN404

① $E \rightarrow E + E / id$

Grammar

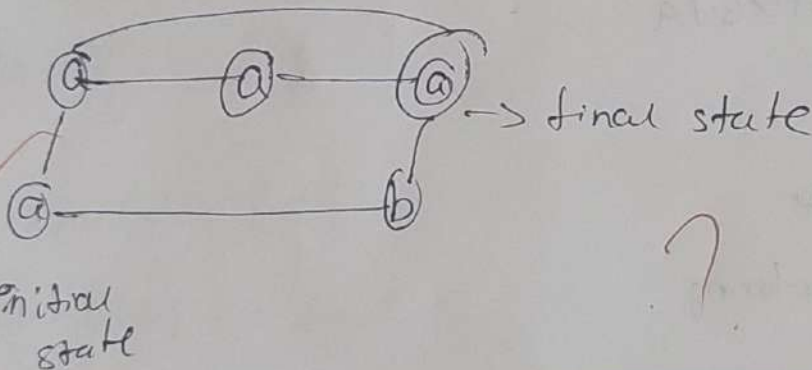
$E \rightarrow E + E / id$

$E \rightarrow E + E$

$E \rightarrow id$

67
100

2) $a(a|b)^*a$

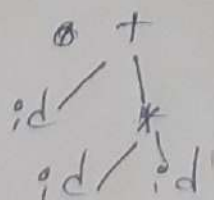


③ Construct a parse tree for the string $id + id * id$

i) lexical analysis

$\langle id \rangle \langle + \rangle \langle id \rangle \langle * \rangle \langle id \rangle$

Constructing parse tree



4)

$$A \rightarrow A\alpha / B$$

$$A \rightarrow A\alpha$$

$$A \rightarrow B$$

$$A' \rightarrow \alpha A$$

$$A' \rightarrow \epsilon$$

$$A' \rightarrow \alpha A' / \epsilon$$

α

5)

$$S \rightarrow i\epsilon t s / i\epsilon t s e s / a$$

$$\epsilon \rightarrow b$$

$$A \rightarrow \alpha B / \alpha$$

After left factoring

$$A \rightarrow \alpha A$$

$$A' \rightarrow B / \alpha$$

$$S \rightarrow i\epsilon t s s /$$

$$S' \rightarrow e s / \epsilon$$

$$C \rightarrow b //$$

first and follow

6A $S \rightarrow aBDh$

$B \rightarrow c \mid \epsilon$

$C \rightarrow bc \mid \epsilon$

$D \rightarrow \epsilon \mid f$

$E \rightarrow g \mid \epsilon$

$F \rightarrow f \mid \epsilon$

$\text{first}(S) = \{a\}$ ✓

$\text{first}(B) = \{c\}$ ✓

$\text{first}(C) = \{b, \epsilon\}$ ✓

$\text{first}(D) = \{g, \epsilon\}$ ✓

$\text{first}(E) = \{g, f, \epsilon\}$ ✓

$\text{first}(F) = \{g, \epsilon\}$ ✓

$\text{follow}(S) = \{\$ \}$ ✓

$\text{follow}(B) = \{g, f, \$ \}$

$\text{follow}(C) = \{g, f, \$ \}$

$\text{follow}(D) = \{g, f, \$ \}$ ✓

$\text{follow}(E) = \{f, g, \$ \}$

$\text{follow}(F) = \{f, g, \$ \}$

7 Given grammar

$S \rightarrow sa \mid sb \mid cd$

$S \rightarrow sa$

grammar containing recursion

$$S \rightarrow sa$$

$$S \rightarrow sb$$

$$S \rightarrow bs' / \epsilon$$

$$S' \rightarrow as' / \epsilon$$

3



8 $E \rightarrow TE', E' \rightarrow +TE' / \epsilon,$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

2 $\text{first}(E') = \{+, \epsilon\}$



9. $S \rightarrow (S)S / \epsilon$



10.

There are four types of actions

1. ~~shift~~ shift

Buffer on the stack

2. reduce

production rule on the stack

3. Accept

The symbol '\$' the ip string is accepted

4. Error

shift (or) reduce error

11. Given the following grammar

Grammar: $S' \rightarrow S$

$S \rightarrow c c$

$C \rightarrow c C | d$

Item: $[S' \rightarrow S]$

Closure

$S' \rightarrow \cdot S$

$S \rightarrow \cdot c c$

$C \rightarrow \cdot c C | d$

12 construct first set of non terminals

$S \rightarrow AB, A \rightarrow a, B \rightarrow b$

$S \rightarrow ash/ab,$

key input ϵ aabb

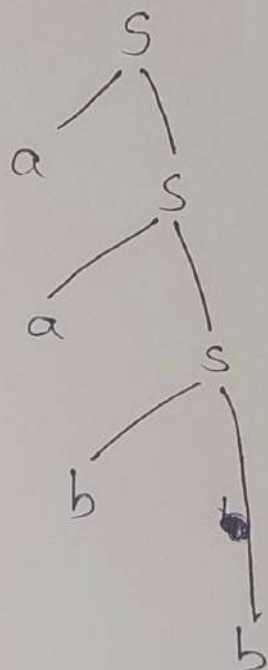
$first(S) = \{a, b\}$

$first(A) = \{a\}$

$first(B) = \{b\}$

$S \rightarrow ash$

$S \rightarrow ab$



136

first and follow

$S \rightarrow A$

$A \rightarrow aB/Ad$

$B \rightarrow b$

$C \rightarrow g$

$$\text{first}(S) = \{a\}$$

$$\text{first}(A) = \{a\}$$

$$\text{first}(A') = \{d, \epsilon\}$$

$$\text{first}(B) = \{b\}$$

$$\text{first}(C) = \{g\}$$

$$\text{follow}(S) = \{\$ \}$$

$$\text{follow}(A) = \{\$ \}$$

$$\text{follow}(A') = \{\$ \}$$

$$\text{follow}(B) = \{d, \$ \}$$

$$\text{follow}(C) = \{d, \$ \}$$

14

Lex code

1. $\{$

include <stdio.h>

1. $\}$

%. %.

[a-zA-Z]+

[0-a]+

[1+ln)

%. %.

Parse expression

$E \rightarrow ST$

$E \rightarrow ST$

or

$T \rightarrow nm$

$T \rightarrow id$

15a given grammar

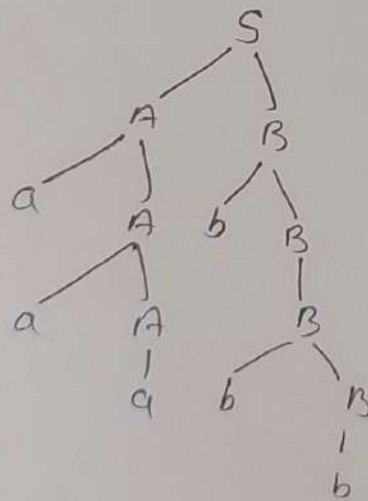
$S \rightarrow AB$

$A \rightarrow aA \mid a$

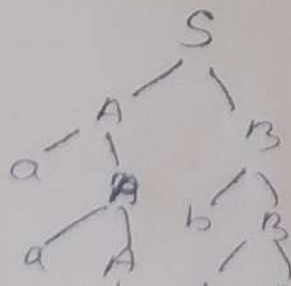
$B \rightarrow bB \mid b$

Input string = $aaabbb$

LMD



RMD



2

$$S \rightarrow AB$$

$$A \rightarrow aA/\epsilon$$

Non terminal follow

$$B \rightarrow bB/c$$

$$\text{first}(S) = \{a, b, c\}$$

$$\text{follow}(S) = \{\$ \}$$

$$\text{first}(A) = \{a, \epsilon\}$$

$$\text{follow}(A) = \{b, \epsilon\}$$

$$\text{first}(B) = \{b, c\}$$

$$\text{follow}(B) = \{\epsilon\}$$

left recursion

$$A \rightarrow A\alpha/\beta$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A'/\epsilon$$

left factoring

$$A \rightarrow \alpha B_1/\alpha B_2$$

$$A \rightarrow \alpha A'$$

$$A' \rightarrow B_1/B_2$$

18

$$E \rightarrow TE'$$

$$E' \rightarrow +TE'/\epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT'/\epsilon$$

$$F \rightarrow (E)/id$$

$$\text{first}(E) = \{ (, id \}$$

$$\text{first}(E') = \{ +, \epsilon \}$$

$$\text{first}(T) = \{ (, id \}$$

$$\text{first}(T') = \{ *, \epsilon \}$$

$$\text{first}(F) = \{ (, id \}$$

$$\text{Follow}(E) = \{ \$,) \}$$

$$\text{Follow}(E') = \{ \$,) \}$$

$$\text{Follow}(T) = \{ +, \$,) \}$$

$$\text{Follow}(T') = \{ +, *, \$,) \}$$

$$\text{Follow}(F) = \{ *, +, \$ \}$$

Parse table:

non-terminal		id	+	*	(	)	\$
E	$E \rightarrow TE'$				$E \rightarrow TE'$		
E'			$E' \rightarrow +TE$			$E' \rightarrow TE$ $E' \rightarrow TE'$	
T	$T \rightarrow FT'$				$T \rightarrow FT'$		
T'			$T' \rightarrow \epsilon$			$T' \rightarrow \epsilon$ $T' \rightarrow \epsilon$	
F	$F \rightarrow id$				$F \rightarrow (e)$		

Stacks implementation

Stack	input	output
\$	id + id * id \$	$E \rightarrow TE'$
\$ E' T	id + id * id \$	$E \rightarrow TE'$
\$ E' T F	id + id * id \$	$T \rightarrow FT'$
\$ E' T F	+ id * id \$	$F \rightarrow id$
\$ E'	id * id \$	$T' \rightarrow \epsilon$
\$ E'	id * id \$	$E' \rightarrow TE'$

stack	input	output
\$E' T' F +	id * id \$	F → id
\$T' id	* id \$	(-1 → E
\$		
\$T' F +	id \$	T' → * FT'
\$T' id	\$	T' → * FT'

11 $E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

input string id + id * id (id)

stack	input	output
\$	id + id * id (id) \$	shift
\$E	+ E * E (E) \$	shift
\$E+	E * E (E) \$	shift
\$E+E	E * E (E) \$	shift
\$E*	E (E) \$	$E \rightarrow E + E$ Reduce
\$E * E	(E) \$	shift
\$E(	E) \$	$E \rightarrow E * E$ Reduce
\$E(E	) \$	shift
\$E(E)	\$	shift
\$	\$	Reduce $E \rightarrow (E)$

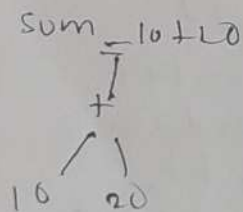

```

20 #include <stdio.h>
    int main() {
        int a=10, b=20;
        int sum=a+b;
        printf("%d", sum);
        return 0;
    }

```

- lexical analyzer : source code converts into tokens
- Syntax analyzer : That tokens are convert into parse tree
- Semantic : verify variables
- Intermediate code : Three address code memory
- code optimizer : it reduces the codes
- code generator : converts into machine codes

b) $sum = a + b$



8

c) $sum = a + b$

$S \rightarrow id = E$

$E \rightarrow E + T$

$T \rightarrow id / num$

Input string = $sum = a + b$

